Guía Gamificada deU1T2 25 Minutos

Modelado con PlantUML: 2 diagramas de casos de uso nivel 0 + 1 diagrama de clases de análisis

Caso: aplicativo móvil para seguimiento a egresados ESPE – IASA-I

Dirigido a	Estudiantes de 6to nivel de la carrera de Software con base previa en Ingeniería de Requisitos.
Modalidad	Trabajo colaborativo en equipos de 3 a 4 integrantes.
Producto final	Dos diagramas de casos de uso nivel 0 y un diagrama de clases de análisis construidos con sintaxis PlantUML.

Idea central del reto: traducir requisitos funcionales del proyecto a modelos UML claros, trazables y coherentes, usando PlantUML como herramienta de representación textual.

1. Propósito del taller

Adaptar la guía gamificada al modelado de artefactos de análisis para el proyecto de seguimiento a egresados. El equipo trabajará como si fuera una célula de análisis responsable de transformar necesidades del caso en modelos UML iniciales listos para discusión técnica.

2. Base conceptual del reto

El caso del proyecto plantea una necesidad institucional: mantener contacto continuo con egresados, actualizar información personal, difundir oportunidades, apoyar capacitación y recopilar retroalimentación para la mejora académica.

- Desde Ingeniería de Requisitos, el taller se enfoca en tres preguntas clave:
  - ¿Quién interactúa con el sistema? → actores.
  - ¿Qué objetivos persigue cada actor? → casos de uso.
  - ¿Qué conceptos del dominio participan y cómo se relacionan? → clases de análisis.

3. Objetivos de aprendizaje

Lo que ya conocen	Lo que practicarán hoy	Evidencia esperada
Actores y necesidades	Identificación de actores primarios y secundarios	Listado depurado de actores
Requisitos funcionales	Conversión de RF a casos de uso nivel 0	2 diagramas de CU coherentes
Trazabilidad	Relación entre objetivo del negocio y funcionalidad	Justificación breve por equipo
Análisis de dominio	Extracción de entidades, atributos y relaciones	1 diagrama de clases de análisis

4. Modelos que debe construir cada equipo

- Diagrama 1: Casos de uso nivel 0 orientado al actor Egresado.
- Diagrama 2: Casos de uso nivel 0 orientado al actor Unidad de Seguimiento a Graduados / Administrador.
- Diagrama 3: Diagrama de clases de análisis del dominio principal del sistema.

Sugerencia didáctica: cada diagrama debe responder a una parte clara del problema. No se trata de dibujar todo, sino de modelar lo esencial de la primera versión del sistema.

5. Mecánica gamificada

Insignia	Cómo se obtiene	Puntos
Radar de Actores	Identifica actores correctos y evita redundancias.	3
Arquitecto de Casos de Uso	Construye 2 diagramas nivel 0 con nombres claros y relaciones válidas.	5
Modelador del Dominio	Propone clases de análisis pertinentes con relaciones coherentes.	5
Sintaxis PlantUML Segura	Escribe código PlantUML legible y consistente.	3

6. Secuencia del taller (25 minutos)

Tiempo	Fase	Actividad del equipo	Resultado esperado
0-3 min	Activación	Asignar roles: coordinador, analista de requisitos, modelador PlantUML y vocero. Leer el caso base.	Equipo organizado.
3-8 min	Ronda 1	Identificar actores y agrupar funcionalidades por objetivo del usuario.	Mapa de actores y objetivos.
8-14 min	Ronda 2	Construir el primer diagrama de casos de uso nivel 0 orientado al Egresado.	CU nivel 0 – Actor Egresado.
14-19 min	Ronda 3	Construir el segundo diagrama de casos de uso nivel 0 orientado a la Unidad de Seguimiento / Administrador.	CU nivel 0 – Actor institucional.
19-23 min	Ronda 4	Extraer clases del dominio, atributos esenciales y relaciones de análisis.	Diagrama de clases de análisis.

Tiempo	Fase	Actividad del equipo	Resultado esperado
23-25 min	Cierre	Validar coherencia entre requisitos, actores y clases. Exposición breve.	Síntesis y retroalimentación.

7. Desarrollo del reto por rondas

Ronda 1. Radar de actores y objetivos

- ¿Quién usa la app?: El Agricultor, encargado del registro físico de las cosechas en campo.
- ¿Quién administra la información?: El Administrador de la Finca, responsable de la gestión de personal, configuración del sistema y análisis de métricas.
- Objetivos :Sustituir el registro manual en cuadernos por una solución digital que permita la trazabilidad por lotes y el monitoreo en tiempo real de la productividad de la pitahaya.

Ronda 2 y 3. Casos de uso nivel 0

- Se seleccionaron los casos de uso de alto nivel: Gestionar agricultores, Gestionar cuenta e Ingresar producción campo.

Ronda 4. Diagrama de clases de análisis

- Se extrajeron del dominio de la Finca Jirah: Administrador, Agricultor, Producción (gavetas), Lote, Cuenta .

8. Caso base del taller

Caso: La ESPE – IASA-I requiere una app móvil para mantener comunicación con sus egresados. El sistema debe permitir actualización de datos personales, difusión de oportunidades de pasantías, acceso a recursos de capacitación, envío de noticias de alto impacto y recopilación de retroalimentación. Además, la unidad responsable necesita administrar contenido y consultar información de apoyo para la toma de decisiones.

9. Entregable del equipo

- Lista breve de actores identificados.
  - Administrador, agricultor
- Código PlantUML del diagrama de casos de uso nivel 0 – Agricultor.

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor Agricultor

rectangle "Sistema Gestion Agricola (SGA) - Finca Jirah" {
  usecase "Ingresar producción campo" as UC_IngProd
  usecase "Gestionar cuenta" as UC_GestCuenta
}

' Relaciones del Agricultor
Agricultor --> UC_IngProd
Agricultor --> UC_GestCuenta
@enduml
```

- Código PlantUML del diagrama de casos de uso nivel 0 – Administrador.

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor Administrador

rectangle "Sistema Gestion Agricola (SGA) - Finca Jirah" {
  usecase "Gestionar Agricultores" as UC_GestAgri
  usecase "Gestionar cuenta" as UC_GestCuenta
}

' Relaciones del Administrador
Administrador --> UC_GestAgri
Administrador --> UC_GestCuenta
@enduml
```

- Código PlantUML del diagrama de clases de análisis.

```
@startuml
class Usuario {
  - cedula: String
  - nombre: String
  - correo: String
}

class Administrador extends Usuario {
}

class Agricultor extends Usuario {
  - estadoActivo: boolean
}

class Cuenta {
  - nombreUsuario: String
  - contraseniaHash: String
  - temaInterfaz: String
}

class Produccion {
  - id_registro: int
  - pesoGavetaKg: float
  - fechaHora: DateTime
}

class Lote {
  - numeroLote: int
  - descripcion: String
}

' Relaciones de Análisis
Usuario "1" *-- "1" Cuenta : posee
Administrador "1" --> "*" Agricultor : gestiona
Agricultor "1" --> "*" Produccion : registra en campo
Administrador "1" --> "*" Produccion : monitorea
Produccion "*" --> "1" Lote : vinculada a
@enduml
```

- Justificación oral de 30 a 40 segundos sobre la trazabilidad entre requisitos y modelos.

10. Plantilla rápida para completar en clase

Actores principales	Administrador y Agricultor.

Casos de uso del Agricultor	Ingresar producción campo, Gestionar cuenta.
Casos de uso del Administrador	Gestionar Agricultores, Gestionar cuenta.
Clases candidatas del dominio	Administrador, Agricultor, Cuenta, Producción, Lote.
Relaciones principales entre clases	El Administrador gestiona al Agricultor. El Agricultor ingresa la Producción. La Producción pertenece a un Lote. Tanto Administrador como Agricultor poseen una Cuenta.
Observaciones de trazabilidad	Existe una relación directa entre el problema de digitalización en campo y el modelo. El caso de uso "Ingresar producción campo" se mapea a la clase Producción y su relación obligatoria con Lote, lo que garantiza poder tener el histórico y estadística por edad de las plantas, tal como lo requiere el dueño de la finca. Por otro lado, la seguridad y el control de personal se resuelven centralizando el acceso a través de la clase Cuenta y las operaciones del Administrador.

11. Mini sintaxis PlantUML esencial

Ejemplo 1. Caso de uso nivel 0 de administrador

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor Administrador

rectangle "Sistema Gestion Agricola (SGA) - Finca Jirah" {
  usecase "Gestionar Agricultores" as UC_GestAgri
  usecase "Gestionar cuenta" as UC_GestCuenta
}

' Relaciones del Administrador
Administrador --> UC_GestAgri
Administrador --> UC_GestCuenta
@enduml
```

Ejemplo 2. Diagrama de clases de análisis

```
@startuml
class Usuario {
  - cedula: String
  - nombre: String
  - correo: String
}

class Administrador extends Usuario {
}

class Agricultor extends Usuario {
  - estadoActivo: boolean
}

class Cuenta {
  - nombreUsuario: String
  - contraseniaHash: String
  - temaInterfaz: String
}

class Produccion {
  - id_registro: int
  - pesoGavetaKg: float
  - fechaHora: DateTime
}

class Lote {
  - numeroLote: int
  - descripcion: String
}

' Relaciones de Análisis
Usuario "1" *-- "1" Cuenta : posee
Administrador "1" --> "*" Agricultor : gestiona
Agricultor "1" --> "*" Produccion : registra en campo
Administrador "1" --> "*" Produccion : monitorea
Produccion "*" --> "1" Lote : vinculada a
@enduml
```

12. Criterios de evaluación rápida

Criterio	Qué se observa	Puntaje
Comprensión del caso	Los actores y objetivos corresponden al problema del proyecto.	5
Calidad de casos de uso	Los 2 diagramas nivel 0 son claros, consistentes y sin redundancias.	5
Calidad del diagrama de clases	Las clases de análisis representan el dominio y sus relaciones esenciales.	5
Uso de PlantUML y comunicación	La sintaxis es legible y la explicación del equipo es técnica y breve.	5

13. Cierre sugerido del docente

Mensaje de cierre: modelar bien no es dibujar mucho, sino representar con precisión lo que el sistema debe hacer y los conceptos clave que lo sostienen. Si los requisitos están claros, PlantUML permite convertirlos en artefactos trazables, discutibles y técnicamente útiles.

Documento adaptado para el caso de seguimiento a egresados ESPE – IASA-I y orientado al uso de PlantUML en Análisis y Diseño de Software.